



MÓDULO 4 · AULA 29

Aula 29

Qualidade e validação de dados

Testar o DADO em si — com Great Expectations, pandera e Altair AI Studio.

Instrutor: Douglas Adriano Queiroz

O que vamos ver hoje



Qualidade de dados

As dimensões que importam



O que validar

Schema, faixas, nulos



Great Expectations

Testar o dado como código



pandera

Schema de DataFrame



Altair AI Studio

Visual, sem código (RapidMiner)



Dado que enganou

Casos reais

As dimensões da qualidade de dados



Compleitude

Faltam valores? (nulos onde não deveria)



Unicidade

Há duplicados que deveriam ser únicos?



Validade

Está no formato/faixa certa? (idade 0–120)



Consistência

Bate entre tabelas/colunas? (total = soma)



Atualidade

O dado está atualizado o suficiente?



Acurácia

Reflete a realidade? (o CEP existe?)

Validar o dado é testar o dado

Assim como testamos o código, escrevemos 'expectativas' sobre o dado e as verificamos automaticamente.

Schema: as colunas e tipos esperados existem? (preço é número?)

Faixas: os valores estão dentro do previsto? (nota entre 1 e 5)

Nulos e únicos: campos obrigatórios preenchidos? ids sem duplicata?

Na esteira: roda a cada nova carga de dados — barra dado ruim antes de usar.



Analogia da casa

É o inspetor na porta do armazém conferindo cada caixa que chega: peso certo? lacre intacto? validade ok? Caixa fora do padrão não entra. Validação de dados é esse inspetor na entrada do pipeline.



Leve daqui: Dado também tem 'testes': schema, faixas, nulos e duplicados, a cada carga.

Exemplo: schema com pandera

Definimos o formato esperado do DataFrame; se o dado violar, o teste falha.

```
validacao.py

import pandas as pd
from pandera import Column, Check

schema = pd.DataFrameSchema({
    'id': Column(int, unique=True),
    'bairro': Column(str, nullable=False),
    'preco': Column(float, Check.gt(0)), # preço > 0
    'nota': Column(int, Check.in_range(1, 5)), # nota de 1 a 5
})

schema.validate(df) # lança erro se algum valor violar as regras
```



Leve daqui: Great Expectations e pandera transformam 'regras do dado' em testes automáticos.

Exemplo: as mesmas regras com Great Expectations

No Great Expectations você declara 'expectativas' — o próprio nome do método já diz o que ele verifica.

```
validacao_ge.py

in port great_expectations as gx

validador = gx.from_pandas(df) # cria um validador a partir do DataFrame

# declara EXPECTATIVAS sobre o dado:
validador.expect_column_n_values_to_be_unique('id')
validador.expect_column_n_values_to_not_be_null('bairro')
validador.expect_column_n_values_to_be_between('nota', 1, 5)

resultado = validador.validate() # roda todas as expectativas
print(resultado['success'])      # True se todas passaram
# o GE ainda gera um relatório visual (Data Docs) do que passou/falhou
```



Leve daqui: Os métodos 'expect_...' são legíveis até para quem não é de dados — e viram um relatório visual.

pandera x Great Expectations



pandera

- Schema declarativo e leve, em Python.
- Ótimo dentro do código e do CI (rápido).
- Foco em validar DataFrames pandas.



Great Expectations

- Mais completo; gera relatórios visuais (Data Docs).
- Roda sobre bancos, Snowflake e Spark (escala).
- Ótimo para pipelines de dados grandes.



Leve daqui: pandera para checagens rápidas no código; Great Expectations para validar dados em escala com relatórios.

Altair AI Studio: validação visual (sem código)

Para quem não programa, o Altair AI Studio (antigo RapidMiner) faz limpeza, validação e modelagem arrastando 'operadores'.

Operadores: blocos visuais para ler, filtrar, validar tipos e detectar faltantes.

Qualidade: detecta valores ausentes, outliers e inconsistências no dataset.

Validação cruzada: avalia o modelo (Cross Validation) sem escrever código.

No desafio da trilha: importe o processo .rmp pronto e rode o pipeline de churn (Aula 31).



Analogia da casa

É como montar a esteira da fábrica com peças de encaixe: cada estação (operador) faz uma checagem no produto (dado) antes de passar adiante — tudo visual, sem parafusar nada.



Leve daqui: Altair AI Studio valida e modela de forma visual — mesma ideia, sem escrever código.

De RapidMiner a Altair AI Studio (a herança)

A ferramenta que você instala hoje como 'Altair AI Studio' é o antigo RapidMiner — só mudou de dono e de nome.

Origem (2001): nasceu na Universidade de Dortmund (Alemanha) como 'YALE'; virou RapidMiner em 2007.

Altair (2022): a Altair comprou o RapidMiner e o integrou ao seu portfólio de dados e IA.

Siemens (2025): a Siemens adquiriu a Altair (~US\$ 10 bi); a ferramenta passou a se chamar Altair AI Studio.

Na prática: o motor é o mesmo — os processos .rmp continuam abrindo normalmente.



Analogia

É como uma padaria de bairro querida que foi comprada por uma rede grande: mudou a placa e o dono, mas o pão (o RapidMiner que você aprende) é o mesmo. Saber a história ajuda a não se perder quando o nome muda.



Leve daqui: Altair AI Studio = RapidMiner com novo nome (Altair → Siemens). O que você aprende continua valendo.

Quando o dado ruim passou

Falta de validação de dados gera relatórios e decisões erradas em silêncio.



Unidade errada

Misturar metros e pés, reais e centavos, ou datas em formatos diferentes distorce todo o resultado. Uma checagem de faixa/formato pega isso na entrada.



Duplicatas e nulos

Registros duplicados inflam totais; campos nulos quebram cálculos (média vira erro). Validar unicidade e completude evita o relatório errado.



Leve daqui: A maioria dos erros de análise vem do dado, não do gráfico — valide na entrada.



Resumão da aula



Qualidade de dados: completude, unicidade, validade, consistência, atualidade, acurácia.



Validar o dado = escrever expectativas e verificá-las automaticamente.



Great Expectations e pandera testam schema, faixas, nulos e duplicados.



Altair AI Studio (antigo RapidMiner) valida e modela de forma visual (sem código).



Rode a validação a cada carga — barre o dado ruim antes de usá-lo.



A maioria dos erros de análise nasce no dado; valide na entrada.



Glossário da aula

Qualidade de dados — O quanto o dado é confiável para decidir

Schema — Estrutura esperada (colunas, tipos, regras)

Great Expectations — Ferramenta para validar dados como testes

pandera — Validação de schema de DataFrame em Python

Completeness — Ausência de valores faltantes indevidos

Unicidade — Sem duplicatas onde deve ser único

Validade — Valores no formato/faixa esperados

Outlier — Valor muito fora do padrão

Altair AI Studio — Plataforma visual de dados (antigo RapidMiner)

Cross Validation — Avaliar o modelo em várias divisões dos dados



Referências e para aprofundar

Great Expectations / pandera. greatexpectations.io e union.ai/pandera — validação de dados.

Docs oficiais. pandas.pydata.org · scikit-learn.org · docs.pytest.org · greatexpectations.io · rapidminer.com

Trilha do curso: Data Science e Testes — Projeto Social Garapuvu 2026.